

Лабораторная по БД №3

2025-04-10 14:16

Status: #baby

Tags: Базы данных Лабы

Лабораторная по Базам данных №3

Задание.

Для отношений, полученных при построении предметной области из лабораторной работы №1, выполните следующие действия:

- Опишите функциональные зависимости для отношений полученной схемы (минимальное множество);
- Приведите отношения в 3NF (как минимум). Постройте схему на основеNF (как минимум).
- Опишите изменения в функциональных зависимостях, произошедшие после преобразования в 3NF (как минимум). Постройте схему на основеNF;
- Преобразуйте отношения в BCNF. Докажите, что полученные отношения представлены в BCNF. Если ваша схема находится уже в BCNF, докажите это;
- Какие денормализации будут полезны для вашей схемы? Приведите подробное описание.

Придумайте триггер и связанную с ним функцию, относящиеся к вашей предметной области, согласуйте их с преподавателем и реализуйте на языке PL/pgSQL.

Отчёт по лабораторной работе должен содержать:

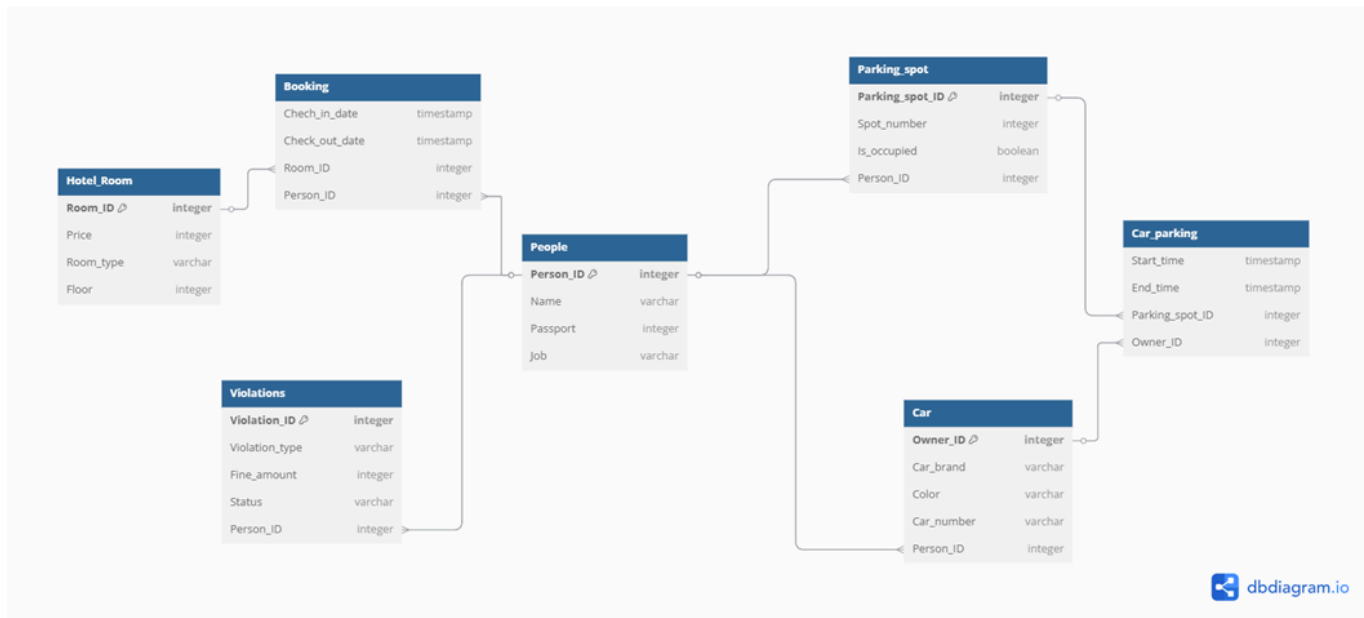
1. Текст задания.
2. Исходная, нормализованная и денормализованная модели.
3. Ответы на вопросы, представленные в задании.
4. Функция и триггер на языке PL/pgSQL
5. Выводы по работе.

Темы для подготовки к защите лабораторной работы:

1. Нормализация. Формы
2. Функциональные зависимости. Виды

3. Денормализация
4. Язык PL/pgSQL

Отношения из предметной области лабораторной №1:



1. ФЗ:

Функциональную зависимость (ФЗ) обозначают $A \rightarrow B$. Обратим внимание, что A и B могут быть не только единичными атрибутами, но и группами, которые составлены из нескольких атрибутов одного отношения.

Говорят, что функциональные зависимости являются связями типа «один-ко-многим», которые существуют внутри отношения.

Транзитивная функциональная зависимость. Пусть A, B, C – атрибуты какого-либо отношения. При этом $A \rightarrow B$ и $B \rightarrow C$ и отсутствует обратное соответствие, то есть $C \nrightarrow B$ и $B \nrightarrow A$. В таком случае C транзитивно зависит от A .

Многозначная зависимость. Пусть A, B, C – атрибуты некоторого отношения R . В данном отношении R существует многозначная зависимость $R: A \twoheadrightarrow B$. В лишь в том случае, когда множество значений B , которое соответствует паре значений A и C , зависит только от A и не зависит от C .

ключи - атрибуты столбцов, которые однозначно (уникально) определяют запись в БД;

функциональные зависимости - ограничения между двумя взаимосвязанными атрибутами;

нормальные формы - этапы для достижения определенного качества БД.

Person_ID → Name, Passport, Job (тривиальная фз)

Violation_ID → Violation_type, Fine_amount, Status, Person_ID (тривиальная фз)

Owner_ID → Car_brand, Color, Car_number, Person_ID (условие в 1 лабе было, что у каждого человека только 1 машина)

Parking_spot_ID → Spot_number, Is_occupied, Person_ID

Room_ID → Price, Room_type, Floor

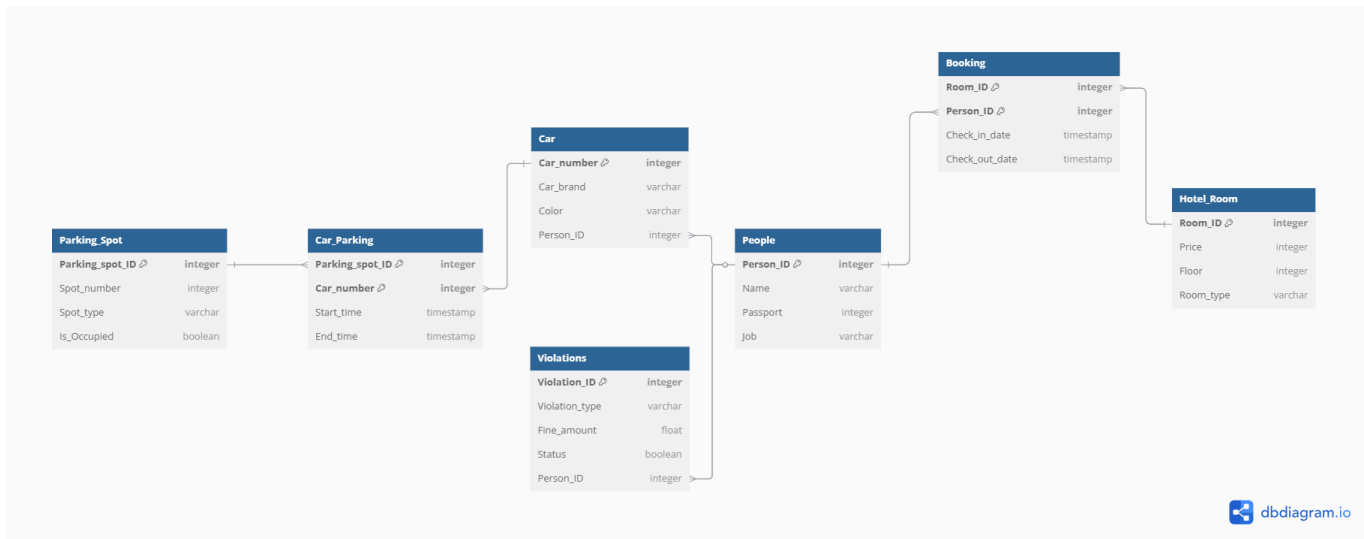
(Check_in_date, Room_ID, Person_ID) → Check_out_date (нетривиальная фз)

(Start_time, Parking_spot_ID) → End_time, Owner_ID (нетрив фз)

2. Отношения в 3NF:

Стадия	Аномалии избыточности
Ненормализованная (нулевая) форма (UNF)	Это состояние перед любой нормализацией. В таблице присутствуют избыточные и сложные значения
Первая нормальная форма (1NF)	Разбиваются повторяющиеся и сложные значения; все экземпляры становятся атомарными
Вторая нормальная форма (2NF)	Частичные зависимости разделяются на новые таблицы. Все строки функционально зависимы от первичного ключа
Третья нормальная форма (3NF)	Транзитивные зависимости разбиваются на новые таблицы. Не ключевые атрибуты зависят от первичного ключа
Нормальная форма Бойса-Кода (BCNF)	Транзитивные и частичные функциональные зависимости для всех потенциальных ключей разбиваются на новые таблицы
Четвертая нормальная форма (4NF)	Удаляются многозначные зависимости
Пятая нормальная форма (5NF)	Удаляются JOIN-зависимости (зависимости соединения)

Почти все отношения в моей базе данных в 3NF, но возможна транзитивная зависимость в таблице Car через Person_ID, поэтому можно заменить первичный ключ Owner_ID на Car_number, так как второе значение тоже уникально.



3. Ф3:

Person_ID → Name, Passport, Job

Violation_ID → Violation_type, Fine_amount, Status, Person_ID

Car_number → Car_brand, Color, Person_ID

Parking_spot_ID → Spot_number, Spot_type, Is_Occupied

(Parking_spot_ID, Car_number) → Start_time, End_time

(Room_ID, Person_ID) → Check_in_date, Check_out_date

Room_ID → Price, Floor, Room_type

4. Отношения в BCNF

Что нужно было привести в BCNF:

```

Table Car_Parking {
  Parking_ID integer [primary key]
  Parking_spot_ID integer
  Car_number integer
  Start_time timestamp
  End_time timestamp
}
  
```

Машина может парковаться на одном месте многократно, нужен уникальный ID.

Остальное:

Person_ID — ключ, и это единственная зависимость.

Violation_ID — ключ.

Car_number — ключ.

Parking_spot_ID — ключ.

Room_ID - ключ.

Parking_spot_ID - ключ.

5. Денормализации БД

Объединение People и Car

Если часто нужно получать ФИО владельца машины.

Можно добавить в Car поля Name, Passport, Job.

Только при смене данных человека нужно обновлять все его машины

Добавить Name в Violations

Если часто нужно выводить список нарушений с ФИО.

Можно добавить Name в Violations.

Добавить данные о машине в Car_Parking

Чтобы избежать JOIN с Car при просмотре парковки.

Можно добавить в Car_Parking: Car_brand, Color.

Добавить Room_type и Price в Booking

Если часто нужна информация о номере при анализе бронирований.

Можно добавить в Booking: Room_type, Price.

6. Триггер

*Триггер — это сочетание хранимой в базе данных процедуры и события, которое заставляет ее выполняться. Такими событиями могут быть: ввод новой строки таблицы, изменение значений одного или нескольких ее столбцов и (или) удаление строки таблицы.

Триггеры также делятся на три основных типа.

- Триггеры DML активизируются предложениями ввода, обновления и удаления информации (INSERT, UPDATE, DELETE) до или после выполнения предложения, на уровне строки или таблицы.
- Триггеры замещения (instead of) можно создавать только для представлений (либо объектных, либо реляционных). В отличие от триггеров DML, которые выполняются в дополнение к предложениям DML, триггеры замещения выполняются вместо предложений DML, вызывающих их срабатывание. Триггеры замещения должны быть строковыми триггерами. Часть VI. Создание приложений на SQL 324
- Системные триггеры активизируется не на предложение DML, выполняемое над таблицей, а на системное событие, например, на запуск или останов базы данных. Системные триггеры срабатывают и на предложения DDL, такие как создание таблицы. Триггеры также подразделяются на строковые (строчные или уровня строки таблицы) и табличные (операторные или уровня всей таблицы). Строковый триггер запускается один раз для каждой строки,

обрабатываемой активизирующим предложением. Табличный триггер выполняется только один раз, независимо от того, сколько строк содержит запускающее его предложение.

Триггер для моей предметной области:

Автоматически обновлять флаг **Is_Occupied** в таблице **Parking_Spot** при добавлении записи в **Car_Parking**

- Когда машина паркуется — поле **Is_Occupied** в **Parking_Spot** становится **true**.
- Когда машина уезжает — **Is_Occupied** становится **false**.

```
CREATE OR REPLACE FUNCTION update_parking_spot_status()
RETURNS TRIGGER AS $$
BEGIN
IF TG_OP = 'INSERT' THEN
UPDATE Parking_Spot
SET Is_Occupied = true
WHERE Parking_spot_ID = NEW.Parking_spot_ID;

ELSIF TG_OP = 'DELETE' THEN
UPDATE Parking_Spot
SET Is_Occupied = false
WHERE Parking_spot_ID = OLD.Parking_spot_ID;
END IF;

RETURN NULL;
END;
```

```
CREATE TRIGGER trg_update_parking_status
AFTER INSERT OR DELETE ON Car_Parking
FOR EACH ROW
EXECUTE FUNCTION update_parking_spot_status();
```

Защита лабы

1. Нормализация. Формы

Цель нормализации: исключить избыточное дублирование данных, которое является причиной аномалий, возникших при добавлении, редактировании и удалении кортежей(строк таблицы).

Первая нормальная форма

Отношение находится в 1НФ, если все его атрибуты являются простыми, все

используемые домены должны содержать только скалярные значения. Не должно быть повторений строк в таблице.

Вторая нормальная форма

Отношение находится во 2НФ, если оно находится в 1НФ и каждый не ключевой атрибут неприводимо зависит от Первичного Ключа(ПК).

Неприводимость означает, что в составе потенциального ключа отсутствует меньшее подмножество атрибутов, от которого можно также вывести данную функциональную зависимость.

Третья нормальная форма

Отношение находится в 3НФ, когда находится во 2НФ и каждый не ключевой атрибут нетранзитивно зависит от первичного ключа. Проще говоря, второе правило требует выносить все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы в отдельные таблицы.

Нормальная форма Бойса-Кодда (НФБК) (частная форма третьей нормальной формы)

Для отношений, имеющих один первичный ключ, НФБК является 3НФ.

Отношение находится в НФБК, когда каждая нетривиальная и неприводимая слева функциональная зависимость обладает потенциальным ключом в качестве детерминанта.

Четвертая нормальная форма

Отношение находится в 4НФ, если оно находится в НФБК и все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от ее потенциальных ключей.

Пятая нормальная форма

Отношения находятся в 5НФ, если оно находится в 4НФ и отсутствуют сложные зависимые соединения между атрибутами.

2. Функциональные зависимости. Виды вверху

3. Денормализация

Денормализация базы данных — это процесс систематического объединения данных для быстрого получения информации. Этот процесс снижает уровень нормальных форм, что приводит к уменьшению целостности данных.

С другой стороны, производительность извлечения данных увеличивается. Вместо выполнения множества дорогостоящих операций JOIN на многочисленных

таблицах, денормализация помогает объединить данные, которые часто или логически взаимосвязаны.

4. Язык PL/pgSQL

очев